Prompts

- → Static Prompts
- → Dynamic Prompts

Ⓐ Static Prompts ⇒ Here, we send hardcoded messages to the LLM. (For Eg. "Write a poem about life").

Problem: Relying on users to provide the full prompt is often risky. They might make spelling errors, skip instructions or hallucinate facts.

Ⓑ Dynamic Prompts ⇒ Here, we use a template, so that the user can fill in specific variables (like specific topic or styles), while the developer controls the structure.

Solution: This ensures the LLM always receives a well-structured command, regardless of user skill-level.

Ques. Why can't we directly use the 'f-strings' to provide prompts to LLMs instead of using a {Prompt Template} class?

Ans ⇒ ① Validation ⇒ It automatically checks if all input variables are present. If a variable is missing or an extra one is added, it throws an error immediately, preventing run times failures later.

② Reusability ⇒ We can save templates as JSON files and load them into different files or projects, keeping our main code clean.

③ Ecosystem Integration ⇒ It fits perfectly into LangChain's "Chain" (pipelines), allowing us to link prompts and models seamlessly, which f-string can't do easily.

④ Message Types (Roles) ⇒ When building chatbots (Multi-turn conversation) LangChain distinguishes between three message types.

- → System Message ⇒ The "Rule Setter". It is sent at the very start to define AI's behaviour (eg. "You are a doctor").
- → Human Message ⇒ The user's input query.
- → AI Message ⇒ The response generated by the model.

Why? Tagging messages helps the LLM to distinguish better who said what, maintaining context over long conversations.

Scanned with OKEN Scanner

Chat Prompt Template: Similar to Prompt Template, but designed for 'Chat Models' (list of messages) rather than simple string LLMs. It allows us to create a dynamic conversation structure, such as fixed system message combined with a dynamic human message based on user input.

Message Placeholder: The major problem is in a chatapp, we don't know how long the conversation will be. So, we cannot hardcode 5 or 10 message slots in our templates.

Solution: "Message Placeholder" creates a dynamic slot in our prompt template that can expand to fit the entire chat history (memory) retrieved from a database. It inserts the list of previous messages between the System Message and the latest Human Message.

Structured Output in LangChain

Standard LLMs responses are plain texts. While we humans, understand this, it is difficult for code/machines to process it programmatically. Thus, we need to force the LLMs to return the response in some standard/specific format (like dict, JSON, etc.) This enables LLMs to talk to other systems (Databases, APIs) on its own and not just limited to humans. Some key use-case of standard output are —

(a) Data Extraction ⇒ Converting raw data into database entries. Eg: Uploading a resume and automatically extracting Name, Education and Previous Company and save it to a database.

(b) Building APIs ⇒ Creating services that process text and return JSON. Eg: An API that takes a product review and returns 'Sentiment', 'Pros' and 'Cons' for a front-end application.

(c) Agents ⇒ Agents use 'tools' like Search APIs which require specific inputs/(arguments), not conversational text. Structured output bridges the gap between the LLM's reasoning and tool's input requirements.

Scanned with OKEN Scanner

* with_structured_output() ⇒ This is a universal LangChain function with models that natively support structured output like (OpenAI and Gemini). In this, we have to define a "Schema" (a blueprint of the data we want), pass it to this function, and the model attempts to generate data matching that structure.

⇒ There are three different ways using which we can define schemas and tell the LLM what structure we want in the output.

(A) TypedDict (Native Python): It gives a standard Python way to hint at dictionary keys and value types. It is built-in and has a simple syntax. The only problem is there is no implicit validation. For Eg: If the model returns a string instead of the integer, the code will not throw any error, which can lead to bugs later.
It is best to be used in case of simple internal projects where rigorous validation isn't critical.

(B) Pydantic: It is a data validation library which keeps validating the data. It can handle data type validation, constraints, default values, type-coersion, etc. on its own. It is the best approach to be used for creating production systems and building reliable APIs.

(C) JSON: It is a language-agnostic format and it is a universal standard that all the applications/softwares follow. It should be used for projects involving various languages (eg. A Python backend talking to a JavaScript frontend) where we need a shared structure.

Modes: When using 'with_structured_output()', we can often specify a method: (i) JSON mode ⇒ General JSON return (often used with Gemini/Claude)
(ii) Function calling ⇒ Often the default for OpenAI; highly effective for Agents.

Limitations: Not all the models support this function. For Eg. Open-Source models downloaded from Hugging Face may fail with this function because they aren't trained for using this function.

Scanned with OKEN Scanner